Local Search

Heuristic Optimization Techniques 192.162, VU 6 ECTS

Enrico Iurlano, Günther Raidl, and Laurenz Tomandl
heuopt@ac.tuwien.ac.at

Algorithms and Complexity Group
Institute of Logic and Computation, TU Wien

WS 2026

Version from October 15, 2026



Local Search: Trying to Improve a Solution

Local search

We want to improve a given initial solution, e.g., obtained from a construction heuristic.

Basic idea: Check similar solutions in a neighborhood of the current one,
move to a better (or equally good) solution, and repeat (Aarts and Lenstra, 2003).

procedure local search
begin
  $x \leftarrow$ initial solution;
  repeat:
    choose an $x' \in N(x)$;
    if $f(x') \leq f(x)$ **then**
      $x \leftarrow x'$;
  until stopping criterion satisfied;
end

- $f(x)$ is the objective function, $N(x)$ is the neighborhood of x

(We assume minimization.)

Decisions to be made for a local search

- Solution representation, i.e., data structure used for solutions
- Neighborhood structure, i.e., which solutions are considered to be neighbors?
- Step function that defines
  - the way the neighborhoods are searched and
  - which neighbor is selected to continue with.
- Stopping criterion: When shall the algorithm terminate?

Neighborhood structure

Definition (Neighborhood structure)

A neighborhood structure is a function $N : S \to 2^S$ that assigns to each solution $x \in S$ a set of neighbors $N(x) \subseteq S$.

For a solution $x \in S$, $N(x)$ is then the neighborhood of x.

Neighborhood structure

Definition (Neighborhood structure)

A neighborhood structure is a function $N : S \to 2^S$ that assigns to each solution $x \in S$ a set of neighbors $N(x) \subseteq S$.

For a solution $x \in S$, $N(x)$ is then the neighborhood of x.

- Neighborhood structures are typically defined via move operations, i.e., small changes that can be applied to the current solution.
- Visualization possible as neighborhood graph
- Trade-off to consider: size of neighborhood vs. complexity of the search
  - Larger neighborhoods have higher chances to contain superior solutions but are more time-consuming to check.

Example: Maximum Satisfiability Problem (MAX-SAT)

Given: m clauses $C = (C_k)_{k=1,\dots,m}$ on n Boolean variables $x = (x_1, x_2, \dots, x_n) \in \{0,1\}^n$

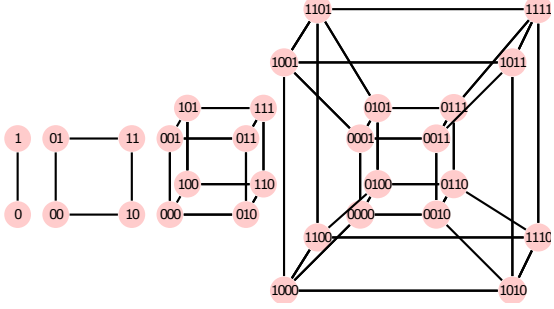
Goal: Find a variable assignment maximizing the number of satisfied clauses.

Example: $C_1 = x_2 \vee x_3$, $C_2 = \overline{x_1} \vee x_2 \vee x_4 \vee \overline{x_5}$, $C_3 = \overline{x_1} \vee \overline{x_2} \vee x_3$

- Natural solution representation: binary vector x
- What is an intuitive neighborhood structure?

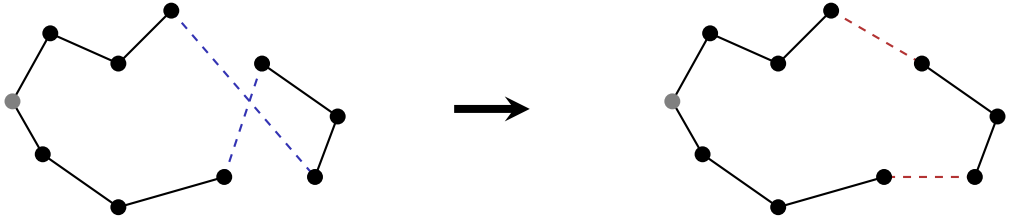
k-flip neighborhood for binary vectors

- k-flip neighborhood structure: Neighboring solutions have a Hamming distance $\leq k$, i.e., up to k variables are flipped.
- Neighborhood size: $O(n^k)$
- Hypercubes of dimension n represent the 1-flip neighborhood:



Example: symmetric TSP

- Solution representation: set of edges
- k-exchange (k-opt) neighborhood:
  neighboring solutions differ in up to k edges – and are still feasible tours(!)

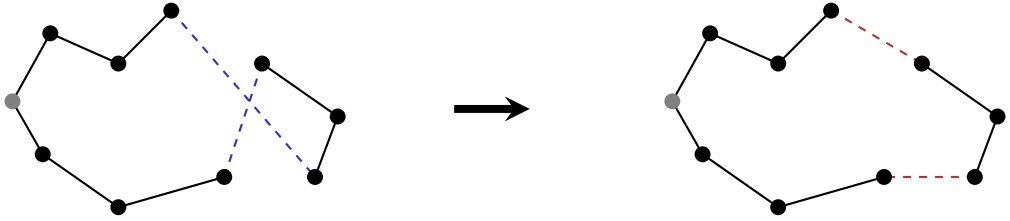


Example: 2-exchange move for the TSP

- $k \geq 2$

Example: symmetric TSP

- Solution representation: set of edges
- k-exchange (k-opt) neighborhood:
  neighboring solutions differ in up to k edges – and are still feasible tours(!)

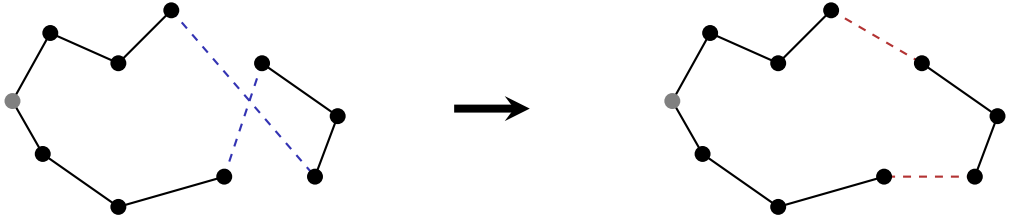


Example: 2-exchange move for the TSP

- $k \geq 2$
- Neighborhood size: What do you think?

Example: symmetric TSP

- Solution representation: set of edges
- k-exchange (k-opt) neighborhood:
  neighboring solutions differ in up to k edges – and are still feasible tours(!)



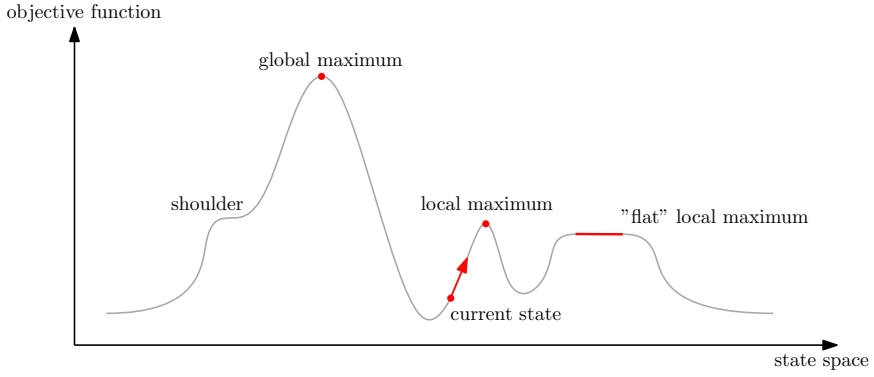
Example: 2-exchange move for the TSP

- $k \geq 2$
- Neighborhood size: What do you think? $O(n^k)$
  - more specifically $\binom{n}{k}$ possibilities to remove k edges,
  - then $O(k!)$ possibilities to reconnect obtained k paths to a cycle

Local vs. global Optima

Definition (Local optimum)

A solution $\hat{x} \in S$ is a local optimum (maximum/minimum) w.r.t. neighborhood $N(\hat{x})$ if there is no solution $x \in N(\hat{x})$ that is better than $\hat{x}$,
i.e., $\forall x \in N(\hat{x}) : \; f(\hat{x}) \geq f(x)$ resp. $f(\hat{x}) \leq f(x)$.

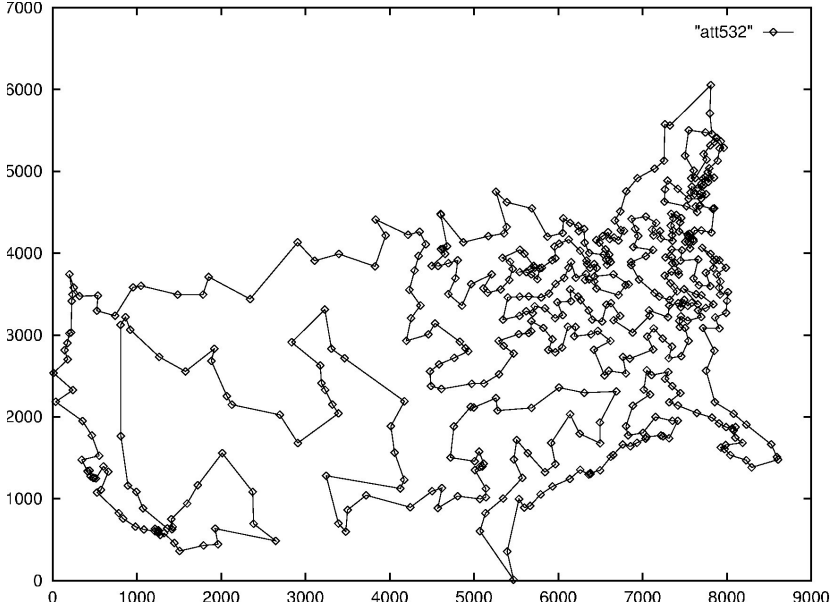


We call $\hat{x}$ a strict local optimum if there also is no equally good solution in $N(x)$.

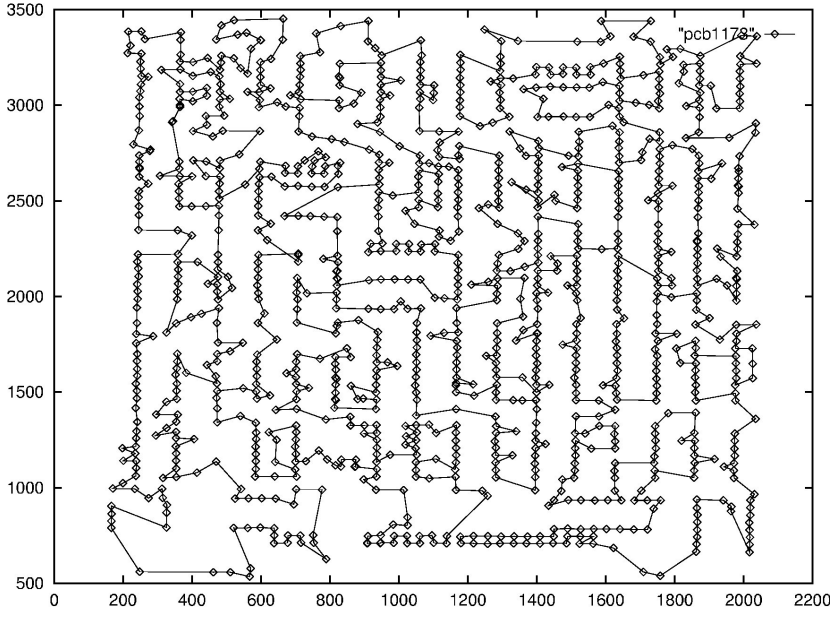
Example: Local search for the TSP

- In general, local search is not guaranteed to find a global optimum,
  as it cannot improve upon any local optimum w.r.t. the used neighborhood structure.
- k-opt solution: a locally optimal solution w.r.t. the k-exchange neighborhood.

2-opt solution for TSP (1)



(Almost) 2-opt solution for TSP (2)



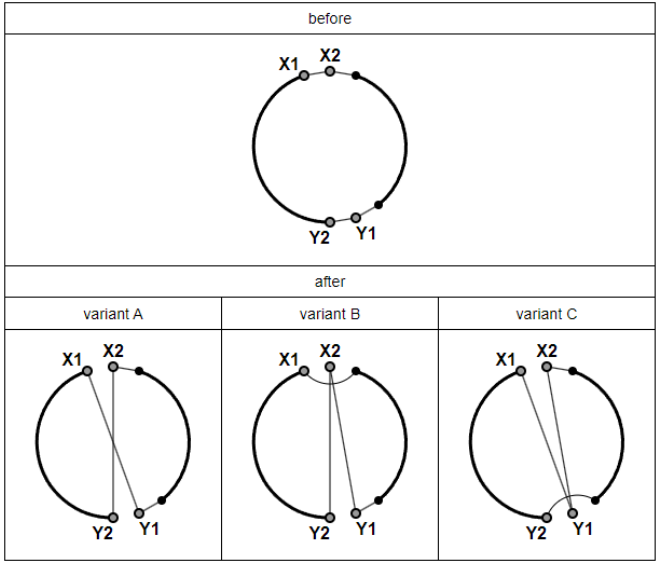
Exampe: Local search for the TSP

- 2-opt local search commonly used, often comes within 6–8% of optimal.
- 3-opt local search often comes within 3–4%, but already much more time consuming.
- 4-opt too time-consuming for practical purposes.
- Other neighborhood structures for TSP, in particular the asymmetric TSP:
  - Move one node to another position – size $O(n^2)$
  - Move a subsequence to another position – size $O(n^3)$
    often restricted to subsequences of small length $\rightarrow$ size $O(n^2)$

Example: Local search for the TSP

- Sometimes done:
  Enriching 2-opt with a small subset of promising 3-opt or 4-opt moves

  "2.5-opt" considers combination of 2-exchange and move of single node:



Step function in local search – choosing $x' \in N(x)$

Best improvement:

Search through $N(x)$ *completely* and take a best neighbor.

First improvement:

Search $N(x)$ in a systematic order, take first solution that is better than x.

Random neighbor:

From $N(x)$ select just a single neighboring solution at random.

The selection process has substantial influence on the performance,
but no strategy is always better than the others!

Typically random neighbor is much faster, but many more local search iterations are needed.

Runtime of local search

Runtime of a 2-opt local search for the TSP with best improvement step function?

- Completely searching one 2-exchange neighborhood takes $O(n^2)$ time.
  - Important: Delta-evaluation of neighbors! (To come...)
- How many iterations needed to reach a local optimum?

Runtime of local search

Runtime of a 2-opt local search for the TSP with best improvement step function?

- Completely searching one 2-exchange neighborhood takes $O(n^2)$ time.
  - Important: Delta-evaluation of neighbors! (To come...)
- How many iterations needed to reach a local optimum?
  - Worst case: O(n!) (without proof)
    $\rightarrow$ local search has exponential worst case runtime!
  - But: In practice also reasonable fast on (very) large instances!

Stopping criteria for local search (and metaheuristics more generally)

What stopping criteria may be meaningful?

Stopping criteria for local search (and metaheuristics more generally)

Local optimum reached:

Of course, continuing does not make sense.

But how to detect? $\rightarrow$ easy when using best/first improvement

Time limit:

Stop search if a time limit is exceeded.

Solution quality:

Stop search if the current solution is good enough

e.g., close enough to a known bound, or sufficient for a practical application

Iteration limit

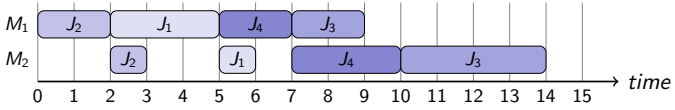
Stop search after a fixed number of iterations.

Convergence:

Stop search when solution did not improve (by more than some ε) over a certain number of last iterations.

Often a combination of several stopping criteria is applied.

Example: Flow Shop Problem with linear permutation representation



Recap: Flow Shop Problem

Given: n jobs $J_1, \ldots, J_n$ and m machines $M_1, \ldots, M_m$

Each job has to be processed sequentially *on every machine*
in the given order of the machines.
A machine can only process one order at a time in an uninterrupted fashion.

$z_{jk} \geq 0$ processing durations for each job j on each machine k

Goal: Find a schedule, such that the sum of finishing times is minimized.

A solution x can be represented as a permutation of the job indices $\{1, 2, \ldots, n\}$:
$x = (\pi_1, \pi_2, \ldots, \pi_n)$

- E.g., $x = (2, 4, 1, 3)$ for $n = 4$

Linear permutations – neighborhoods

Swap neighbors operator: swap the ith element with the $(i+1)^{th}$ element.

$$N_S(x) = \{\delta_S^i(x) : i = 1, \ldots, n\}$$

$$\delta_S^i(\pi_1 \ldots \pi_i \pi_{i+1} \ldots \pi_n) = (\pi_1 \ldots \pi_{i+1} \pi_i \ldots \pi_n)$$

Interchange operator: swap the ith element with the jth element.

$$N_X(x) = \{\delta_X^{ij}(x) : 1 \leq i < j \leq n\}$$

$$\delta_S^i(\pi) = (\pi_1 \ldots \pi_{i-1} \pi_j \pi_{i+1} \ldots \pi_{j-1} \pi_i \pi_{j+1} \ldots \pi_n)$$

Move subsequences of size k: e.g., k=1

$$N_I(x) = \{\delta_I^{ij}(x) : i = 1, \ldots, n, j = 1, \ldots, n, j \neq i\}$$

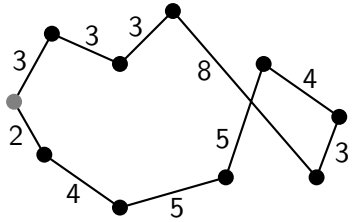
$$\delta_I^{ij}(\pi) = \begin{cases} (\pi_1 \ldots \pi_{i-1} \pi_{i+1} \ldots \pi_j \pi_i \pi_{j+1} \ldots \pi_n) & \text{if } i < j \\ (\pi_1 \ldots \pi_j \pi_i \pi_{j+1} \ldots \pi_{i-1} \pi_{i+1} \ldots \pi_n) & \text{if } i > j \end{cases}$$

Delta-Evaluation

(Incremental Evaluation)

Delta-evaluation

Given: A (suboptimal) solution to the following
EUCLIDEAN SYMMETRIC TSP:

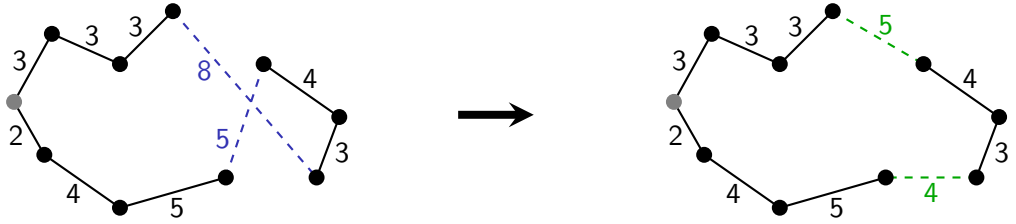


Starting at the **gray node** at the left side and going clockwise, we compute the objective value of the solution x:

$$f(x) = 3 + 3 + 3 + 8 + 3 + 4 + 5 + 5 + 4 + 2 = 40$$

Delta-evaluation

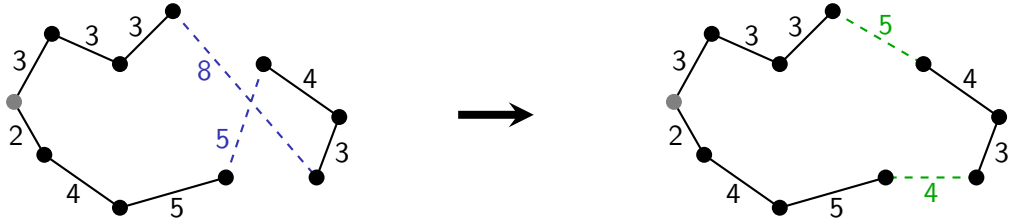
Now we apply a **2-opt-move** (swap blue and green edges):



Question: How would we compute the new objective value $f(x')$?

Delta-evaluation

Now we apply a **2-opt-move** (swap blue and green edges):



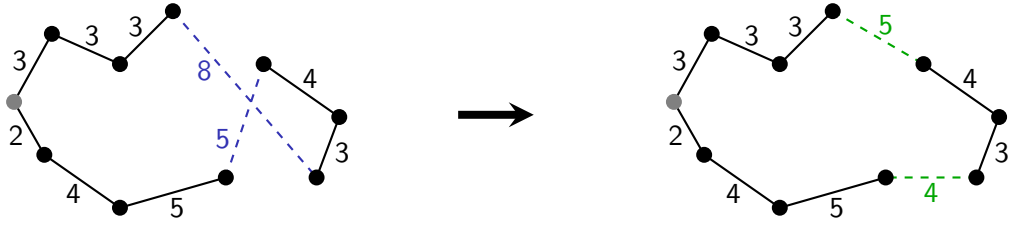
Question: How would we compute the new objective value $f(x')$?

Inefficient: Compute $f(x')$ from scratch

$$f(x') = 3 + 3 + 3 + 5 + 4 + 3 + 4 + 5 + 4 + 2 = 36$$

Delta-evaluation

Now we apply a **2-opt-move** (swap blue and green edges):



Question: How would we compute the new objective value $f(x')$?

Inefficient: Compute $f(x')$ from scratch

$$f(x') = 3 + 3 + 3 + 5 + 4 + 3 + 4 + 5 + 4 + 2 = 36$$

Better: Derive $f(x')$ from $f(x)$ just considering the changes ("delta")

$$f(x') = f(x) - (8 + 5) + (5 + 4) = 40 - 13 + 9 = 36 \quad (1)$$

Computational complexity: $O(1)$ instead of $O(n)$

Elements of delta-evaluation

The idea behind **Delta-Evaluation** (also known as **Incremental Evaluation**):

Recompute only those parts of a solution that depend on changes!

Elements of delta-evaluation

The idea behind **Delta-Evaluation** (also known as **Incremental Evaluation**):

Recompute only those parts of a solution that depend on changes!

The **performance** highly depends on several aspects:

- Choose an appropriate representation of a solution.
- Possibly use additional data structures for efficient delta-evaluation.
- Make use of caching in the evaluation, i.e.,
  - save important intermediate data/results
  - and invalidate only parts that have changed when performing a move;
  - only recompute parts that were invalidated.
- Optimize the search order in the neighborhood!

Example: Maximum Satisfiability Problem (MAX-SAT)

Given: m clauses $C = (C_k)_{k=1,\dots,m}$ on n Boolean variables $x = (x_1, x_2, \dots, x_n) \in \{0,1\}^n$

Goal: Find a variable assignment maximizing the number of satisfied clauses.

Example: $C_1 = x_1 \vee x_2$, $C_2 = x_1 \vee \overline{x_3}$, $C_3 = \overline{x_2} \vee \overline{x_3}$

- Natural solution representation: binary vector x

Example MAX-SAT: designing a data structure

The following **questions** are interesting during the solution process:

1. How many clauses are currently satisfied?
2. Which clauses are currently satisfied?
3. How many literals are true within each clause?

Example MAX-SAT: designing a data structure

The following **questions** are interesting during the solution process:

1. How many clauses are currently satisfied?
2. Which clauses are currently satisfied?
3. How many literals are true within each clause?

Task: Design a data structure that allows us to answer the above questions efficiently after applying a **1-flip** (without recomputing the computation graph from scratch).

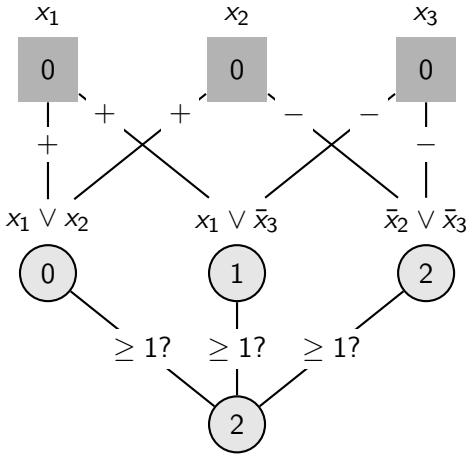
Question: What can such a data structure look like?

Example MAX-SAT: designing a data structure

Solution representation: a binary vector $x = (x_1, x_2, \ldots, x_n)$.

Computation graph:

- Connect each variable x_i with each clause C_j in which the variable is contained; mark edge with '+' if x_i is contained and with '−' if $\overline{x_i}$ is contained.
- Store for each clause the number of true-literals
- Store the number of satisfied clauses.



binary vector $(x_i)_{i \in \{1,\ldots,n\}}$

current assignment

connections between variables and clauses

clauses $(C_j)_{j \in \{1,\ldots,m\}}$

number of true-literals

number of satisfied clauses

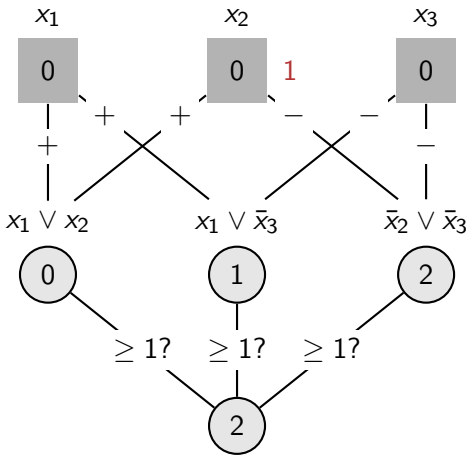
Example MAX-SAT: applying a 1-flip

If we apply a **1-flip** to a variable x_i, we only need to update

- the true-literal-numbers of the clauses connected to x_i, and
- the number of satisfied clauses, if the true-literal-number of a clause changes from 0 to ≥ 1 or vice versa.

Example MAX-SAT: applying a 1-flip

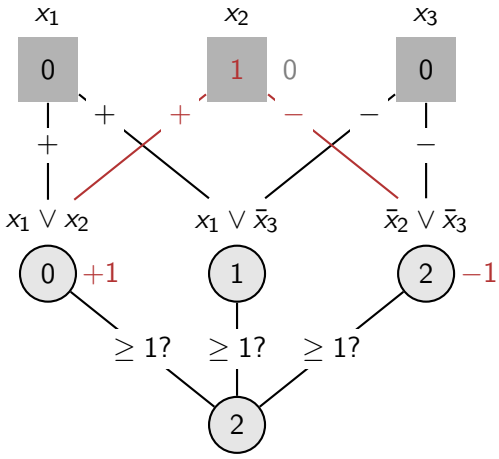
We apply a **1-flip** to variable x_2 (change from 0 to 1).



flip variable x_2

Example MAX-SAT: applying a 1-flip

We apply a **1-flip** to variable x_2 (change from 0 to 1).

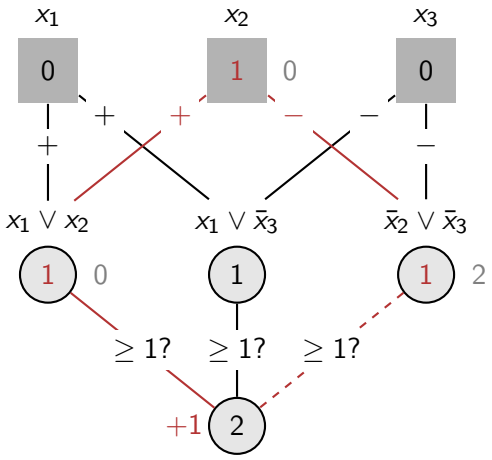


flip variable x_2

update number of true-literals

Example MAX-SAT: applying a 1-flip

We apply a **1-flip** to variable x_2 (change from 0 to 1).



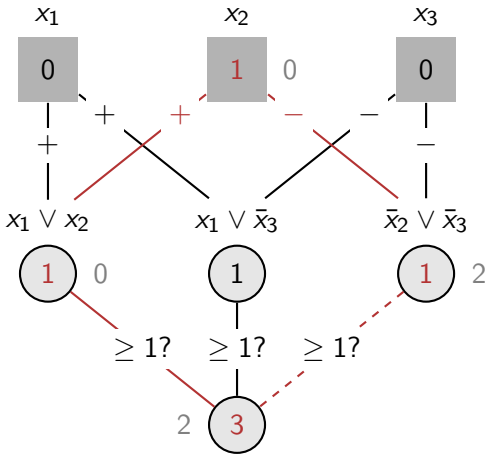
flip variable x_2

update number of true-literals

update number of satisfied clauses

Example MAX-SAT: applying a 1-flip

We apply a **1-flip** to variable x_2 (change from 0 to 1).



flip variable x_2

update number of true-literals

update number of satisfied clauses

Example MAX-SAT

The runtime of updating the objective value after a single **1-flip** thus depends only on the number of clauses in which the affected variable occurs, which is in $O(m)$.

Note: The solution $x = (0, 1, 0)$ is a **local optimum**.

Searching the 2-flip neighborhood structure
We chain above approach:

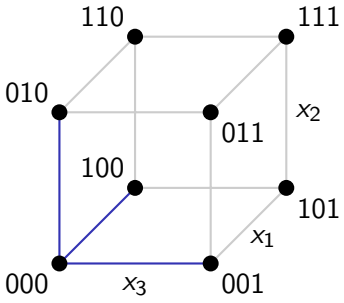
- Flip the first variable and update the solution.
- Flip the second variable and update the solution.

Open question: In which order should we search the neighborhood?

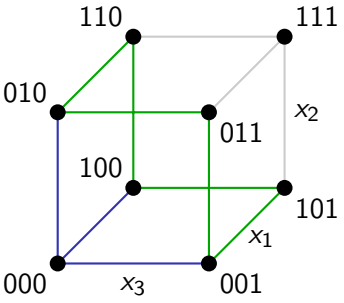
Example MAX-SAT: search order

The search order describes the order in which the solutions of the neighborhood are searched.

1-flip-neighborhood of (0, 0, 0)



2-flip-neighborhood of (0, 0, 0)



Example MAX-SAT: search order

Now we want to explore the **2-flip neighborhood**.

- starting from $x = (0, 0, 0)$

Question: Which of these two search strategies is better?

x_1	x_2	x_3
0	0	0
0	0	1
0	1	0
1	0	0
0	1	1
1	0	1
1	1	0

x_1	x_2	x_3
0	0	0
0	0	1
0	1	1
0	1	0
1	1	0
1	0	0
1	0	1

The left strategy first performs all **1-flips** and then all **2-flips**.

Example: MAX-SAT: search order

Now we want to explore the **2-flip neighborhood**.

- starting from $x = (0, 0, 0)$

Question: Which of these two search strategies is better?

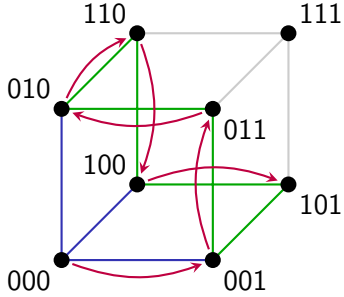
x_1	x_2	x_3	
0	0	0	
0	0	1	1
0	1	0	2
1	0	0	2
0	1	1	3
1	0	1	2
1	1	0	2

x_1	x_2	x_3	
0	0	0	
0	0	1	1
0	1	1	1
0	1	0	1
1	1	0	1
1	0	0	1
1	0	1	1

Answer: The right one, since fewer flips are necessary.

Example: MAX-SAT: search order

x_1	x_2	x_3	
0	0	0	
0	0	1	) 1
0	1	1	) 1
0	1	0	) 1
1	1	0	) 1
1	0	0	) 1
1	0	1	) 1



We only need 6 flips to explore the whole **2-flip-neighborhood**!

Bibliography

E. Aarts and J. K. Lenstra. *Local Search in Combinatorial Optimization*. Princeton University Press, 2003.